

Store Rewards & Points DApp

Technical Documentation

v1.0.0

Blockchain · Solidity · Web3.py

Project Overview

Store Rewards & Points DApp is a full-stack decentralized application (DApp) designed to modernize customer reward systems using blockchain technology. The application allows stores to manage reward items, distribute custom reward coins, and securely track all reward-related activities on-chain using smart contracts.

The system replaces traditional reward cards with a transparent, decentralized reward platform running on an Ethereum-compatible blockchain. Users can earn and redeem Reward Point Coins for store rewards, while administrators manage items, balances, and system controls through protected admin functions.

The application supports both a command-line terminal interface and blockchain-based automation scripts for administration and testing.

1.1 Core Concept

- Store reward items and redemption activities are permanently stored on the blockchain.**
- Users receive and spend Reward Point Coin (RPC), a custom ERC-20 token used inside the system.**
- Only authorized administrators can add reward items, mint coins, pause the system, or transfer ownership.**
- The application uses smart contract access control with onlyOwner protection for all sensitive operations.**
- Blockchain history is scanned to generate activity reports, transaction summaries, and balance snapshots.**
- Users can register personal profiles linked to their wallet addresses and track their activity history on-chain.**

Layer	Technology	Purpose
Blockchain	Solidity + Ganache	Smart contracts and local Ethereum blockchain
Backend	Python 3.8+	Scripts, terminal app, automation, blockchain interaction
Blockchain Client	Web3.py	Communication between Python and blockchain
Smart Contract Standard	ERC-20 (Custom)	Reward Point Coin (RPC)
Interface	Python Terminal CLI	User and admin interaction
Storage	Blockchain + Local Files	Contract state and deployment data
Testing	Python Automated Scripts	Security testing and ownership validation

Smart Contract Design

The application uses two Solidity smart contracts that work together to manage reward items, user interactions, and blockchain-based reward points.

StoreRewards.sol

The primary smart contract responsible for managing store reward items and redemption operations.

Main Responsibilities

- Store and manage reward items
- Handle reward redemption logic

- Register user profiles
- Enforce admin-only operations
- Pause and resume system activity
- Support ownership transfer

Main Functions

- addOrUpdateRewardItem()
- batchSaveRewardItems()
- redeemReward()
- registerUser()
- pause()
- resume()
- transferOwnership()
- getAdmin()

Features

- Uses onlyOwner modifier for sensitive functions
- Stores reward items using structs and mappings
- Includes emergency pause protection
- Validates all user inputs with require
- Maintains on-chain user registration records

RewardPointCoin.sol

A custom ERC-20 token contract representing the system reward currency (Reward Point Coin / RPC).

Main Responsibilities

- Manage user token balances
- Mint new reward points

- Transfer reward points between users
- Provide balance tracking functionality

Main Functions

- mint()
- transfer()
- balanceOf()
- approve()
- transferFrom()

Features

- ERC-20 compatible implementation
- Only the admin can mint new tokens
- Emits Transfer events for blockchain tracking
- Used as the payment mechanism for reward redemption

store_rewards_points_dapp/

```

└─ admin/
|   └─ deploy_and_seed.py    # Deploy contracts and insert sample reward
items
|   └─ admin_dashboard.py    # Blockchain dashboard and analytics
|   └─ admin_helpers.py      # Shared Web3 and deployment helper functions
|   └─ batch_operations.py    # Batch reward item management scripts
|
└─ contracts/
|   └─ RewardPointCoin.sol    # Custom ERC-19 reward token contract
|   └─ StoreRewards.sol       # Main reward management smart contract
|
└─ deployments/
```

```

|   └─ deployments.json      # Contract addresses and deployment metadata
|
|   └─ terminal/
|       └─ terminal_app.py    # Main command-line interface for users/admins
|
|       └─ tests/
|           └─ security_test.py    # onlyOwner and RBAC security tests
|           └─ ownership_test.py   # transferOwnership validation tests
|           └─ pause_resume_test.py # Emergency stop testing
|       └─ run_all_tests.py      # Executes all automated test suites
|
|       └─ reports/
|           └─ balances_snapshot.csv # ETH and RPC balance snapshot export
|       └─ monitoring/
|           └─ live_alerts.py      # Blockchain live alert monitoring script
|
|       └─ config.py             # Blockchain configuration and constants
|       └─ requirements.txt      # Python project dependencies
|       └─ README.md            # System documentation and usage guide
└─ directory-tree.txt          # Auto-generated project tree snapshot

```

4. Setup & Installation

4.1 Prerequisites

Requirement	Version	Notes
Python	3.8+	Core runtime for Python scripts and Web3 integration

Ganache	Latest	Must run on http://127.0.0.1:8545
Git	Any	For cloning the repository
Solidity Compiler	0.8.x	Required for smart contract compilation
Node.js (Optional)	Any	Only needed for advanced contract tooling

4.2 Installation Steps

Step 1 — Clone the Repository

```
git clone <repository-url>  
cd idea_store_project_fixed
```

Step 2 — Create and Activate a Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate  
python3 -m venv venv  
source venv/bin/activate
```

Step 3 — Install Python Dependencies

```
pip install -r requirements.txt
```

Step 4 — Start Ganache

Open Ganache GUI
or run:

```
ganache --port 8545
```

The blockchain must run on:

<http://127.0.0.1:8545>

Step 5 — Deploy Smart Contracts and Seed Data

`python admin/deploy_and_seed.py`

This script performs the following tasks:

- Compiles StoreRewards.sol and RewardPointCoin.sol
 - Deploys contracts to Ganache
 - Creates sample reward items
 - Stores deployment addresses in deployments/deployments.json
-

4.3 Running the Application

Run the Terminal Application

`python terminal/terminal_app.py`

The terminal application provides:

- User registration
 - Reward redemption
 - Balance checking
 - Activity history viewing
 - Hidden admin menu access
-

Run the Admin Dashboard

`python admin/admin_dashboard.py`

This dashboard displays:

- Total reward items
- Total minted RPC tokens
- Blockchain transaction counts

- **Most active user addresses**

Run Automated Security Tests

`python tests/security_test.py`

This verifies:

- **onlyOwner protection**
- **Non-admin access blocking**
- **Ownership transfer security**
- **Pause/resume protection**

Run All Test Suites

`python tests/run_all_tests.py`

This executes the complete project testing workflow automatically.

6. Features

6.1 User Features

Feature	Description
User Registration	Users can register a display name linked to their blockchain wallet address
Reward Item Browsing	View all available reward items stored on-chain
Reward Redemption	Redeem reward items using Reward Point Coin (RPC)
RPC Balance Checker	View current Reward Point Coin balance
ETH Balance Checker	View Ethereum balance for any wallet address

Personal Activity History	Scan blockchain history and display user transactions and reward actions
Terminal Menu System	Interact with the application through a command-line interface
Blockchain Transaction Tracking	All reward and user activities are permanently recorded on-chain
Live Reward Status	View reward availability and redemption information in real time
Secure Blockchain Validation	All actions are verified through smart contract rules before execution

6.2 Admin Features

Feature	Description
Add Reward Item	Create new reward items stored on the blockchain
Update Reward Item	Modify reward item information including cost and availability
Batch Reward Operations	Add or update multiple reward items in a single blockchain transaction
Mint RPC Tokens	Create and distribute Reward Point Coin (RPC) to users
Pause System	Temporarily disable user-facing reward operations during emergencies
Resume System	Re-enable system operations after pause mode
Ownership Transfer	Transfer admin privileges securely to another blockchain address
Admin Dashboard	View blockchain analytics, transaction counts, and system statistics

Top User Analytics	Identify the most active blockchain users
CSV Balance Export	Export ETH and RPC balances to CSV reports
Security Testing Suite	Automated tests verifying access control and admin protection
Blockchain Monitoring	Live alert system for tracking new reward activities

6.3 Security Features

Feature	Description
onlyOwner Access Control	Restricts sensitive functions to the contract owner only
Input Validation	Uses require statements to reject invalid blockchain transactions
Emergency Stop Mechanism	pause() and resume() functions control system availability
Ownership Protection	Old admins automatically lose access after ownership transfer
Revert Protection	Unauthorized actions revert automatically on-chain
Automated Security Tests	Python scripts validate role-based access control behavior

6.4 Blockchain & Reporting Features

Feature	Description
Blockchain Activity Scanning	Reads historical blockchain data directly from Ganache

Transaction Analytics	Tracks and summarizes contract interactions
Balance Snapshot Exporter	Generates CSV reports of ETH and RPC balances
Data History Reports	Displays the most frequently redeemed reward items
Live Alert System	Detects new blocks and reward-related blockchain activity
Smart Contract Event Tracking	Uses emitted blockchain events for monitoring and reporting

BY :

- **Menna Ahmed**
- **Basmala Mahmoud**
- **Mavie Refaat**
- **Salma Ahmed**
- **Naira Kassem**
- **Nouran Hassan**